AUST CSE Carnival 6.0

Intra AUST Programming Contest Editorial

Prepared by: AUST CSE Carnival Problem Setters Team

September 28, 2025

Problem A: Series-Parallel Voltage Calculator

Problem Setter: ineffablekenobi

Let the circuit be represented as a weighted undirected graph with nodes 0 and $n-1$ as source and sink, and voltage V applied. Any vertex of degree 2 (except 0 and $n-1$) can be contracted since resistances in series sum linearly. Let junctions be nodes with degree $\neq 2$ or endpoints.

For each junction, traverse its adjacent edges to find chains ending at another junction, summing the resistances along the chain. If multiple chains connect the same junction pair, they are in parallel, and the equivalent resistance is

$$\frac{1}{R_{\text{eq}}} = \frac{1}{R_1} + \frac{1}{R_2} + \dots$$

where $R_1, R_2, \dots$ are the resistances of the individual chains. This reduces the original graph to a single path of junctions connected by equivalent resistors.

Let this path have resistances $R_1, R_2, \dots, R_k$. The total resistance along the path is

$$R_{\text{tot}} = R_1 + R_2 + \cdots + R_k$$

and the total current is

$$I = \frac{V}{R_{\text{tot}}}.$$

If the query node q is a junction, its voltage is

$$V_q = V - I \cdot R_{0 \to q},$$

where $R_{0 \to q}$ is the sum of resistances from node 0 up to q along the path. If q lies inside a contracted chain between junctions s and t, with total chain resistance L and distance d from s, then

$$R_{0 \to q} = R_{0 \to s} + d$$

and we use the same formula for V_q. If $q = 0$, $V_q = V$; if $q = n - 1$, $V_q = 0$.

Time Complexity: $O(n \log n)$
Solution: View

Problem B: Making Equal

Problem Setter: Remon60

We are given two rows of stones, `row1` and `row2`, and we can swap stones between the rows to make the rows equal. To determine feasibility, count the frequency of each distinct stone size across both rows. If the total frequency of any stone size is odd, it is impossible to equalize the rows, so the answer is -1.

For sizes with even total frequency, compute the surplus in each row: if a stone appears more times in `row1` than in `row2`, the excess copies need to move from `row1` to `row2`, and vice versa. Let d_2 contain all stones that need to move from `row1`, and d contain all stones that need to move from `row2`.

To minimize the cost, observe that swapping a stone x from `row1` with a stone y from `row2` costs $\min(x, y)$. It can be cheaper to swap via the smallest stone present in both rows, $mn = \min(\min(\texttt{row1}), \min(\texttt{row2}))$, giving a cost of $\min(2 \times mn, x)$ or $\min(2 \times mn, y)$ instead of a direct swap. We process swaps greedily: pair the largest element from d with the smallest element from d_2 (or vice versa) and add $\min(2 \times mn, \text{element being swapped})$ to the total cost. Repeat until all required swaps are completed.

Finally, the accumulated sum is the minimum cost to make the rows equal.

Time Complexity: $O(n \log n)$ due to sorting the rows and linear passes to compute frequencies and swaps.
Solution: View

Problem C: The Monster's Dice (Easy Version)

Problem Setters: ApuYe, sahaun

In this easy version, the board has size n, there are m snakes/ladders, and exactly one query ($q = 1$). Moreover, for each connection, $L = R$, so each snake or ladder links exactly two distinct cells.

The goal is to reach cell n from the starting cell c using a standard 6-faced dice, minimizing the number of dice rolls. Johan is assumed to have precise control of the dice to achieve exact outcomes, but we want to compute the **minimal number of moves** rigorously.

Approach:

1. **Coordinate Compression:** Given n can be up to 10^9, we cannot simulate all cells. Instead, collect only relevant cells:
    - Starting cell c

- Cell n
- All snake/ladder endpoints
- Cells reachable within +0..6 steps from each relevant cell

Assign each unique cell a compressed ID. This reduces the problem to a graph with size proportional to the number of relevant positions.

2. **Graph Construction:**
   - For each snake/ladder from a cell x to cell y, add an edge of weight 0 ($x \to y$), representing an instantaneous move.
   - For other relevant cells, add edges to the next 6 positions reachable by dice rolls. If a compressed position a_i can reach a_j with a dice roll, add an edge with weight equal to the minimal number of dice rolls required to move from a_i to a_j:

$$\text{weight} = \left\lceil \frac{a_j - a_i}{6} \right\rceil$$

   Here, the symbol $\lceil \cdot \rceil$ denotes *rounded up to the nearest integer*. Only the next 6 positions are considered from each node to reduce the total number of edges while guaranteeing correctness.

3. **Shortest Path Calculation:** Run a shortest path algorithm on the compressed graph from the starting cell to cell n. Since all edge weights are non-negative, this guarantees minimal dice rolls. If n is unreachable, return -1.

Time Complexity: Let M be the number of unique cells after compression (including queries, edges, and their 6 neighbors). Since each node has at most 6 outgoing edges, using a shortest path algorithm gives a complexity of

$$\mathcal{O}(M \log M).$$

Solution: View

Problem D: The Monster's Dice (Medium Version)

Problem Setters: ApuYe, sahaun

In the medium version, the main update from the easy version is handling multiple queries efficiently and reversing the graph.

Let the target be to compute the minimal number of dice rolls to reach cell n from multiple starting cells $c_1, c_2, \ldots, c_q$.

Fix a node n as the target and reverse all edges in the graph. Snake or ladder edges $y \to x$ have weight 0, representing instantaneous movement. Normal edges between consecutive cells are added in the reversed direction with weight equal to the number of dice rolls needed to traverse the distance.

Let a_i and a_j be the original positions of two compressed cells. The weight of the edge from a_j to a_i is

$$\text{weight} = \frac{a_j - a_i + 5}{6}$$

This represents the minimal number of 6-faced dice rolls needed to move from a_i to a_j. The $+5$ ensures rounded-up integer division.

Perform coordinate compression to reduce the graph size. Include only relevant cells:

- All snake/ladder endpoints
- All query starting positions
- $+1$ to $+6$ neighbors around each relevant cell

Assign a compressed ID to each unique position, reducing the graph to M nodes, where M is the number of unique compressed cells.

Mark all the Snake/ladder starting points, so normal dice-roll edges are not added from these nodes. This guarantees mandatory teleportation is respected.

After building the reversed graph, run a shortest path algorithm from the target cell n. This will give you the minimum cost to reach any cell c from cell n in the reversed graph, which is equivalent to the minimum cost to reach cell n from any cell c in the forward graph.

This approach allows processing up to $q \leq 10^4$ queries efficiently with one shortest path algorithm run, instead of running shortest path algorithm q times.

Time Complexity: $O(M \times \log M)$, where M is the number of unique compressed cells.
Solution: View

Problem E: The Monster's Dice (Hard Version)

Problem Setter: sahaun

In the hard version, the key update from the medium version is handling general snakes/ladders that can cover a range $[L, R]$ instead of a single cell. The graph is also optimized using segment trees for range edges.

Let the target be to compute the minimal number of dice rolls to reach cell n from multiple starting cells $c_1, c_2, \ldots, c_q$. Each snake/ladder can transport from a range of cells $[L, R]$ to a fixed cell x with zero dice rolls.

Key Updates

1. **Range Snakes/Ladders:** Instead of just mapping $x \to y$, we now have a range $[L, R] \to x$. To handle this efficiently, we build a **segment tree** on the compressed cell coordinates. Each node in the segment tree represents a subrange of cells, allowing us to add edges from any cell in $[L, R]$ to x in $O(\log M)$ time.
2. **Graph Construction:**

- **Single edges** represent normal dice-roll moves between non-blocked cells. For two compressed cells $a_i < a_j$, the edge weight is

$$\text{weight} = \frac{a_j - a_i + 5}{6}$$

  This represents the minimal number of 6-faced dice rolls (rounded up integer division).
- **Range edges** represent snakes/ladders with $L \leq \text{cell} \leq R \to x$, with weight 0. Added using segment tree queries.

3. **Coordinate Compression:** Only "interesting" cells are included:
   - All snake/ladder endpoints and their +6 neighbors
   - All query starting cells

   This reduces the graph size to M nodes.
4. **Reversed Graph:** We reverse all edges so that a single Dijkstra from the target cell n computes minimal distances to all query starting points in one run.
5. **Distance Computation:** Dijkstra is run on the compressed, reversed graph. The distance array stores minimal dice rolls to reach n from each compressed node. For each query starting cell c, output the distance or -1 if unreachable.

Time Complexity: $O(M \times \log M)$, where M is the number of unique compressed cells.
Solution: View

Problem F: FORTY EIGHT

Problem Setter: Mushfiq

This is a **giveaway problem** meant to be the easiest in the contest. It has **no input** and simply requires printing a string multiple times.

The task is to output "`OPERAND 48`" **exactly 84 times**, each on a **new line**.

There are no computations, conditions, or edge cases. The solution involves a simple loop that repeats the string 84 times.

This problem is primarily included for fun and as a warm-up to the contest.

Problem G: Pizza Slices

Problem Setter: fahmid07

Let the target be to count all distinct sequences of length k on a circular pizza with N slices, where each slice has a topping represented by a lowercase letter. Consider sequences both clockwise and counter-clockwise.

Fix a direction and take the pizza string S of length N. Let each starting index i generate a sequence of length k. If the sequence goes beyond the last slice, wrap around to the beginning. Represent each sequence by a **hash** to allow fast comparisons.

Use a polynomial rolling hash with two different bases (double hashing) to avoid collisions. Precompute powers of the bases and their modular inverses to enable $O(1)$ extraction of any substring hash. For sequences that wrap around, merge the hashes of the two segments using the rolling hash property.

Let $H(i)$ denote the hash of the sequence starting at position i. Compute $H(i)$ for all $0 \leq i < N$, accounting for wrap-around. Then reverse the string and compute the same for the opposite direction. Store all resulting hashes in a set to keep only unique sequences.

The total number of distinct sequences is the size of the set.

This method counts each sequence exactly once because the set automatically handles uniqueness. Hashing ensures that sequences can be compared in $O(1)$ time instead of $O(k)$, and reversing the string handles sequences in the other direction. Wrap-around sequences are handled by merging the hashes of two consecutive segments.

Time Complexity: $O(N)$.
Solution: View

Problem H: Last Digit String

Problem Setter: SajidAbdullah

Let the target be to find the last K digits of the string S, where S is formed by repeatedly multiplying X by Y exactly Z times and recording the rightmost digit of each product.

Fix a test case with integers X, Y, Z, K. Observe that the last digit of a number modulo 10 depends only on the current last digit and the multiplier Y modulo 10. Therefore, the sequence of last digits forms a periodic sequence with a maximum period of 10.

Let the first part of the sequence before the repetition starts be the pre-cycle, and the repeating part be the cycle. Denote the start of the cycle as `cycle_start` and its length as `cycle_len`. Each position i in the string S can then be determined as:

- If $i <$ `cycle_start`, the digit is the i-th element in the pre-cycle.
- Otherwise, the digit is at position `cycle_start` + ($i -$ `cycle_start`) mod `cycle_len` in the cycle.

The total length of S is Z, and we only need the last K digits, i.e., positions $Z - K$ to $Z - 1$. Using the above method, each of these positions can be directly computed from either the pre-cycle or the cycle, without generating the entire sequence.

This approach works because the last-digit sequence modulo 10 is fully determined by X mod 10 and Y mod 10. Once the cycle is identified, indexing into it repeatedly gives the exact last K digits.

The time complexity is $O(K)$ per test case, as we only compute the last K digits. Detecting the cycle takes at most 10 steps. Across all test cases, the total operations are $O(\sum K)$,

which is guaranteed to be $\leq 10^6$. The space complexity is $O(10)$ for the cycle plus $O(K)$ for storing the result.

Solution: View

Problem I: Shadow Area

Problem Setter: pz1971

Let the target be to find the area of the shadow of a triangle on the XY plane when projected from a light source located at point $P(px, py, pz)$.

Fix the triangle vertices $A(ax, ay, az)$, $B(bx, by, bz)$, $C(cx, cy, cz)$. If we project each vertex onto the XY plane along the line connecting it with the light source P, the coordinates of the shadow point $S(sx, sy, 0)$ of a vertex (x, y, z) can be computed as:

$$sx = px + \frac{pz}{pz - z} \cdot (x - px), \quad sy = py + \frac{pz}{pz - z} \cdot (y - py)$$

Set S_A, S_B, S_C as the shadow points of vertices A, B, C respectively. The area of the shadow triangle is obtained using the shoelace formula:

$$\text{area} = 0.5 \cdot |sx_A \cdot (sy_B - sy_C) + sx_B \cdot (sy_C - sy_A) + sx_C \cdot (sy_A - sy_B)|$$

This method ensures correct projection because the light source is guaranteed to be above the triangle ($pz > az, bz, cz \geq 0$), which prevents degenerate cases where the shadow is undefined.

The algorithm computes the shadow coordinates for all three vertices and applies the shoelace formula directly. Each test case requires a constant number of arithmetic operations. Hence, the time complexity is $O(1)$ per test case, and $O(t)$ for t test cases.

Solution: View

Problem J: UCL Final

Problem Setter: _akiib

Let the target be to partition the array of stamina costs into k contiguous segments such that the maximum stamina any player spends is minimized.

Fix a candidate maximum stamina M. Traverse the array of segment costs $a_1, a_2, \ldots, a_n$ and greedily assign consecutive segments to the current player until adding the next segment would exceed M. When that happens, start a new player's segment. Count the total number of players needed to cover all segments under this constraint.

Set cnt $= 1$ and current_sum $= 0$. For each segment a_i:

- If $a_i > M$, this maximum is impossible since no single player can take even one segment.
- If current_sum $+ a_i \leq M$, add a_i to the current player's segment.
- Else, start a new player with a_i and increment cnt.

The function cnt $\leq k$ checks if M is feasible. The optimal maximum stamina is found by performing binary search on M over the range $[\max(a_i), \sum a_i]$. In each step, check feasibility using the greedy partitioning described above. The minimal feasible M is the answer.

This counts the minimal possible maximum stamina because any smaller M would require more than k players or violate the continuous segment rule. The algorithm computes feasibility in linear time per check and searches logarithmically over the sum range.

Time Complexity: $O(n \log(\sum a_i))$. If $k > n$, it is impossible to allocate segments, and the answer is -1.

Solution: View

Problem K: Hasinar Shornomudra

Problem Setter: ishraqfatin7

Consider the prime factorization of N:

$$N = p_1^{x_1} \cdot p_2^{x_2} \cdot \cdots \cdot p_k^{x_k}$$

- If N is prime or a perfect power of a single prime ($k = 1$), no valid division exists → `!YES`.
- otherwise ($k \geq 2$), a division is always possible → `YES`.
- One pile can have size $p_1^{x_1}$.
- Another pile can have size $p_2^{x_2} \cdot \cdots \cdot p_k^{x_k}$.
- Additional piles of size 1 can be added so that the sum equals N.

The LCM of these piles equals N because $p_1^{x_1}$ and the product of the remaining prime powers are coprime.

Implementation: For each N, check divisors i from 2 up to $\sqrt{N}$:

- If i divides N and N/i is not divisible by i, then N has at least two distinct prime factors → `YES`.
- Otherwise → `!YES`.

This approach detects whether N has multiple prime factors efficiently.

Time Complexity: $O(\sqrt{N})$ per test case.

Solution: View

Problem L: Help them!

Problem Setter: outlast_NATE

Let the target be to find the missing number from $N + 1$ consecutive integers when N numbers are given in any order.

Approach:

- Set a variable `start` to the smallest number.
- Traverse from the smallest to the largest number; for each start:
  - If it does not exist in the array, then `start` is the missing number.
  - Otherwise, increment `start` and continue.

This works because consecutive numbers increase by 1, so the first mismatch identifies the missing integer.

Time Complexity: $O(N)$ per test case.
Solution: View